第20章 注　解

注解（也被称为*元数据*）为我们在代码中添加信息提供了一种形式化的方法，使我们可以在稍后某个时刻非常方便地使用这些数据[⊖]。

注解在一定程度上是在把元数据与源代码文件结合在一起，而不是保存在外部文档中这一大的趋势之下所催生的。同时，注解也是对来自像C#之类的其他语言对Java造成的语言特性压力所做出的一种回应。

注解是众多引入到Java SE5中的重要的语言变化之一。它们可以提供用来完整地描述程序所需的信息，而这些信息是无法用Java来表达的。因此，注解使得我们能够以将由编译器来测试和验证的格式，存储有关程序的额外信息。注解可以用来生成描述符文件，甚至或是新的类定义，并且有助于减轻编写“样板”代码的负担。通过使用注解，我们可以将这些元数据保存在Java源代码中，并利用annotation API为自己的注解构造处理工具，同时，注解的优点还包括：更加干净易读的代码以及编译期类型检查等。虽然Java SE5预先定义了一些元数据，但一般来说，主要还是需要程序员自己添加新的注解，并且按自己的方式使用它们。

注解的语法比较简单，除了@符号的使用之外，它基本与Java固有的语法一致。Java SE5内置了三种，定义在**java.lang**中的注解：

- **@Override**，表示当前的方法定义将覆盖超类中的方法。如果你不小心拼写错误，或者方法签名对不上被覆盖的方法，编译器就会发出错误提示[⊜]。
- **@Deprecated**，如果程序员使用了注解为它的元素，那么编译器会发出警告信息。
- **@SuppressWarnings**，关闭不当的编译器警告信息。在Java SE5之前的版本中，也可以使用该注解，不过会被忽略不起作用。

Java还另外提供了四种注解，专门负责新注解的创建。稍后我们将学习它们。

每当你创建描述符性质的类或接口时，一旦其中包含了重复性的工作，那就可以考虑使用注解来简化与自动化该过程。例如在Enterprise JavaBean（EJB）中存在许多额外的工作，EJB3.0就是使用注解消除了它们。

注解的出现，可以替代某些现存的系统。例如XDoclet（参见http://MindView.net/Books/BetterJava的附录），它是一个独立的文档化工具，专门设计用来生成类似注解一样的文档。与之相比，注解是真正的语言级的概念，一旦构造出来，就享有编译期的类型检查保护。注解（annotation）是在实际的源代码级别保存所有的信息，而不是某种注释性的文字（comment），这使得代码更整洁，且便于维护。通过使用扩展的annotation API，或外部的字节码工具类库（稍后你将会看到），程序员拥有对源代码以及字节码强大的检查与操作能力。

20.1 基本语法

在下面的例子中，使用**@Test**对**testExecute()**方法进行注解。该注解本身并不做任何事情，

⊖ Jeremy Meyer来到Crested Butte花了两周的时间和我一起撰写本章，他的帮助弥足珍贵。

⊜ 这无疑是受到C#中类似功能的启发。C#的这项功能源自一个关键字，而不是注解，并且是编译器强制要求的。也就是说，当C#程序员覆盖一个方法时，必须使用**override**关键字，而在Java中，**@Override**是可选择的。

但是编译器要确保在其构造路径上必须有@Test注解的定义。你将在本章中看到，程序员可以创建一个通过反射机制来运行testExecute()方法的工具。

```
//: annotations/Testable.java
package annotations;
import net.mindview.atunit.*;
public class Testable {
  public void execute() {
    System.out.println("Executing..");
  }
  @Test void testExecute() { execute(); }
} ///:~
```

1060

被注解的方法与其他的方法没有区别。在这个例子中，注解@Test可以与任何修饰符共同作用于方法，例如public、static或void。从语法的角度来看，注解的使用方式几乎与修饰符的使用一模一样。

20.1.1 定义注解

下面就是前例中用到的注解@Test的定义。可以看到，注解的定义看起来很像接口的定义。事实上，与其他任何Java接口一样，注解也将会编译成class文件。

```
//: net/mindview/atunit/Test.java
// The @Test tag.
package net.mindview.atunit;
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Test {} ///:~
```

除了@符号以外，@Test的定义很像一个空的接口。定义注解时，会需要一些元注解(meta-annotation)，如@Target和@Retention。@Target用来定义你的注解将应用于什么地方(例如是一个方法或者一个域)。@Rectetion用来定义该注解在哪一个级别可用，在源代码中(SOURCE)、类文件中（CLASS）或者运行时（RUNTIME）。

在注解中，一般都会包含一些元素以表示某些值。当分析处理注解时，程序或工具可以利用这些值。注解的元素看起来就像接口的方法，唯一的区别是你可以为其指定默认值。

没有元素的注解称为标记注解（marker annotation），例如上例中的@Test。

下面是一个简单的注解，我们可以用它来跟踪一个项目中的用例。如果一个方法或一组方法实现了某个用例的需求，那么程序员可以为此方法加上该注解。于是，项目经理通过计算已经实现的用例，就可以很好地掌控项目的进展。而如果要更新或修改系统的业务逻辑，则维护该项目的开发人员也可以很容易地在代码中找到对应的用例。

1061

```
//: annotations/UseCase.java
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface UseCase {
  public int id();
  public String description() default "no description";
} ///:~
```

注意，id和description类似方法定义。由于编译器会对id进行类型检查，因此将用例文档的追踪数据库与源代码相关联是可靠的。description元素有一个default值，如果在注解某个方法时没有给出description的值，则该注解的处理器就会使用此元素的默认值。

在下面的类中，有三个方法被注解为用例：

```
//: annotations/PasswordUtils.java
import java.util.*;

public class PasswordUtils {
  @UseCase(id = 47, description =
  "Passwords must contain at least one numeric")
  public boolean validatePassword(String password) {
    return (password.matches("\\w*\\d\\w*"));
  }
  @UseCase(id = 48)
  public String encryptPassword(String password) {
   return new StringBuilder(password).reverse().toString();
  }
  @UseCase(id = 49, description =
  "New passwords can't equal previously used ones")
  public boolean checkForNewPassword(
    List<String> prevPasswords, String password) {
   return !prevPasswords.contains(password);
  }
} ///:~
```

1062

注解的元素在使用时表现为名-值对的形式，并需要置于**@UseCase**声明之后的括号内。在**encryptPassword()**方法的注解中，并没有给出**description**元素的值，因此，在**UseCase**的注解处理器分析处理这个类时会使用该元素的默认值。

你应该能够想象得到如何使用这套工具来“勾勒”出将要建造的系统，然后在建造的过程中逐渐实现系统的各项功能。

20.1.2 元注解

Java目前只内置了三种标准注解（前面介绍过），以及四种元注解。元注解专职负责注解其他的注解：

@Target	表示该注解可以用于什么地方。可能的ElementType参数包括： **CONSTRUCTOR**：构造器的声明 **FIELD**：域声明（包括**enum**实例） **LOCAL_VARIABLE**：局部变量声明 **METHOD**：方法声明 **PACKAGE**：包声明 **PARAMETER**：参数声明 **TYPE**：类、接口（包括注解类型）或**enum**声明
@Retention	表示需要在什么级别保存该注解信息。可选的**RetentionPolicy**参数包括： **SOURCE**：注解将被编译器丢弃。 **CLASS**：注解在class文件中可用，但会被VM丢弃。 **RUNTIME**：VM将在运行期也保留注解，因此可以通过反射机制读取注解的信息。
@Documented	将此注解包含在Javadoc中。
@Inherited	允许子类继承父类中的注解。

1063

大多数时候，程序员主要是定义自己的注解，并编写自己的处理器来处理它们。

20.2 编写注解处理器

如果没有用来读取注解的工具，那注解也不会比注释更有用。使用注解的过程中，很重要的一个部分就是创建与使用*注解处理器*。Java SE5扩展了反射机制的API，以帮助程序员构造这类工具。同时，它还提供了一个外部工具**apt**帮助程序员解析带有注解的Java源代码。

下面是一个非常简单的注解处理器，我们将用它来读取**PasswordUtils**类，并使用反射机制

查找@UseCase标记。我们为其提供了一组id值，然后它会列出在PasswordUtils中找到的用例，以及缺失的用例。

```
//: annotations/UseCaseTracker.java
import java.lang.reflect.*;
import java.util.*;

public class UseCaseTracker {
  public static void
  trackUseCases(List<Integer> useCases, Class<?> cl) {
    for(Method m : cl.getDeclaredMethods()) {
      UseCase uc = m.getAnnotation(UseCase.class);
      if(uc != null) {
        System.out.println("Found Use Case:" + uc.id() +
          " " + uc.description());
        useCases.remove(new Integer(uc.id()));
      }
    }
    for(int i : useCases) {
      System.out.println("Warning: Missing use case-" + i);
    }
  }
  public static void main(String[] args) {
    List<Integer> useCases = new ArrayList<Integer>();
    Collections.addAll(useCases, 47, 48, 49, 50);
    trackUseCases(useCases, PasswordUtils.class);
  }
} /* Output:
Found Use Case:47 Passwords must contain at least one
numeric
Found Use Case:48 no description
Found Use Case:49 New passwords can't equal previously used
ones
Warning: Missing use case-50
*///:~
```

这个程序用到了两个反射的方法：**getDeclaredMethods()**和**getAnnotation()**，它们都属于**AnnotatedElement**接口（**Class**、**Method**与**Field**等类都实现了该接口）。**getAnnoation()**方法返回指定类型的注解对象，在这里就是**UseCase**。如果被注解的方法上没有该类型的注解，则返回**null**值。然后我们通过调用**id()**和**description()**方法从返回的**UseCase**对象中提取元素的值。其中，**encriptPassword()**方法在注解的时候没有指定**description**的值，因此处理器在处理它对应的注解时，通过**description()**方法取得的是默认值**no description**。

20.2.1 注解元素

标签@**UseCase**由**UseCase.java**定义，其中包含**int**元素**id**，以及一个**String**元素**description**。注解元素可用的类型如下所示：

- 所有基本类型（**int**，**float**，**boolean**等）
- **String**
- **Class**
- **enum**
- **Annotation**
- 以上类型的数组

如果你使用了其他类型，那编译器就会报错。注意，也不允许使用任何包装类型，不过由于自动打包的存在，这算不是什么限制。注解也可以作为元素的类型，也就是说注解可以嵌套，稍后你会看到，这是一个很有用的技巧。

20.2.2 默认值限制

编译器对元素的默认值有些过分挑剔。首先，元素不能有不确定的值。也就是说，元素必须要么具有默认值，要么在使用注解时提供元素的值。

其次，对于非基本类型的元素，无论是在源代码中声明时，或是在注解接口中定义默认值时，都不能以**null**作为其值。这个约束使得处理器很难表现一个元素的存在或缺失的状态，因为在每个注解的声明中，所有的元素都存在，并且都具有相应的值。为了绕开这个约束，我们只能自己定义一些特殊的值，例如空字符串或负数，以此表示某个元素不存在：

1065

```
//: annotations/SimulatingNull.java
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface SimulatingNull {
  public int id() default -1;
  public String description() default "";
} ///:~
```

在定义注解的时候，这算得上是一个习惯用法。

20.2.3 生成外部文件

有些framework需要一些额外的信息才能与你的源代码协同工作，而这种情况最适合注解表现其价值了。像（EJB3之前）Enterprise JavaBean这样的技术，每一个Bean都需要大量的接口和部署来描述文件，而这些都属于“样板”文件。Web Service、自定义标签库以及对象/关系映射工具（例如Toplink和Hibernate）等，一般都需要XML描述文件，而这些描述文件脱离于源代码之外。因此，在定义了Java类之后，程序员还必须得忍受着沉闷，重复地提供某些信息，例如类名和包名等已经在原始的类文件中提供了的信息。每当程序员使用外部的描述文件时，他就拥有了同一个类的两个单独的信息源，这经常导致代码同步问题。同时，它也要求为项目工作的程序员，必须同时知道如何编写Java程序，以及如何编辑描述文件。

假设你希望提供一些基本的对象/关系映射功能，能够自动生成数据库表，用以存储JavaBean对象。你可以选择使用XML描述文件，指明类的名字、每个成员以及数据库映射的相关信息。然而，如果使用注解的话，你可以将所有信息都保存在JavaBean源文件中。为此，我们需要一些新的注解，用以定义与Bean关联的数据库表的名字，以及与Bean属性关联的列的名字和SQL类型。

1066

以下是一个注解的定义，它告诉注解处理器，你需要为我生成一个数据库表：

```
//: annotations/database/DBTable.java
package annotations.database;
import java.lang.annotation.*;

@Target(ElementType.TYPE) // Applies to classes only
@Retention(RetentionPolicy.RUNTIME)
public @interface DBTable {
  public String name() default "";
} ///:~
```

在**@Target**注解中指定的每一个**ElementType**就是一个约束，它告诉编译器，这个自定义的注解只能应用于该类型。程序员可以只指定**enum ElementType**中的某一个值，或者以逗号分隔的形式指定多个值。如果想要将注解应用于所有的**ElementType**，那么可以省去**@Target**元注解，不过这并不常见。

注意，**@DBTable**有一个**name()**元素，该注解通过这个元素为处理器创建数据库表提供表的名字。

接下来是为修饰JavaBean域准备的注解：

```
//: annotations/database/Constraints.java
package annotations.database;
import java.lang.annotation.*;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Constraints {
  boolean primaryKey() default false;
  boolean allowNull() default true;
  boolean unique() default false;
} ///:~
```

```
//: annotations/database/SQLString.java
package annotations.database;
import java.lang.annotation.*;
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface SQLString {
  int value() default 0;
  String name() default "";
  Constraints constraints() default @Constraints;
} ///:~
```

1067

```
//: annotations/database/SQLInteger.java
package annotations.database;
import java.lang.annotation.*;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface SQLInteger {
  String name() default "";
  Constraints constraints() default @Constraints;
} ///:~
```

注解处理器通过**@Constraints**注解提取出数据库表的元数据。虽然对于数据库所能提供的所有约束而言，**@Constraints**注解只表示了它的一个很小的子集，不过它所要表达的思想已经很清楚了。primaryKey()、**allowNull()**和**unique()**元素明智地提供了默认值，从而在大多数情况下，使用该注解的程序员无需输入太多东西。

另外两个**@interface**定义的是SQL类型。如果希望这个framework更有价值的话，我们就应该为每种SQL类型都定义相应的注解。不过作为示例，两个类型足够了。

这些SQL类型具有**name()**元素和**constraints()**元素。后者利用了嵌套注解的功能，将**column**类型的数据库约束信息嵌入其中。注意**constraints()**元素的默认值是**@Constraints**。由于在**@Constraints**注解类型之后，没有在括号中指明**@Constraints**中的元素的值，因此，**constraints()**元素的默认值实际上就是一个所有元素都为默认值的**@Constraits**注解。如果要令嵌入的**@Constraints**注解中的**unique()**元素为**true**，并以此作为**constraints()**元素的默认值，则需要如下定义该元素：

```
//: annotations/database/Uniqueness.java
// Sample of nested annotations
package annotations.database;

public @interface Uniqueness {
  Constraints constraints()
    default @Constraints(unique=true);
} ///:~
```

1068

下面是一个简单的Bean定义，我们在其中应用了以上这些注解：

```
//: annotations/database/Member.java
```

```
package annotations.database;

@DBTable(name = "MEMBER")
public class Member {
  @SQLString(30) String firstName;
  @SQLString(50) String lastName;
  @SQLInteger Integer age;
  @SQLString(value = 30,
  constraints = @Constraints(primaryKey = true))
  String handle;
  static int memberCount;
  public String getHandle() { return handle; }
  public String getFirstName() { return firstName; }
  public String getLastName() { return lastName; }
  public String toString() { return handle; }
  public Integer getAge() { return age; }
} ///:~
```

类的注解**@DBTable**给定了值MEMBER，它将会用来作为表的名字。Bean的属性**firstName**和**lastName**，都被注解为**@SQLString**类型，并且其元素值分别为30和50。这些注解有两个有趣的地方：第一，他们都使用了嵌入的**@Constraints**注解的默认值；第二，它们都使用了快捷方式。何谓快捷方式呢，如果程序员的注解中定义了名为**value**的元素，并且在应用该注解的时候，如果该元素是唯一需要赋值的一个元素，那么此时无需使用名-值对的这种语法，而只需在括号内给出**value**元素所需的值即可。这可以应用于任何合法类型的元素。当然了，这也限制了程序员必须将此元素命名为**value**，不过在上面的例子中，这不但使语义更清晰，而且这样的注解语句也更易于理解：

```
@SQLString(30)
```

1069 处理器将在创建表的时候使用该值设置SQL列的大小。

默认值的语法虽然很灵巧，但它很快就变得复杂起来。以**handle**域的注解为例，这是一个**@SQLString**注解，同时该域将成为表的主键，因此在嵌入的**@Constraints**注解中，必须对**primaryKey**元素进行设定。这时事情就变得麻烦了。现在，你不得不使用很长的名-值对形式，重新写出元素名和**@interface**的名字。与此同时，由于有特殊命名的**value**元素已经不再是唯一需要赋值的元素了，所以你也不能再使用快捷方式为其赋值了。如你所见，最终的结果算不上清晰易懂。

变通之道

可以使用多种不同的方式来定义自己的注解，以实现上例中的功能。例如，你可以使用一个单一的注解类**@TableColumn**，它带有一个**enum**元素，该枚举类定义了STRING、INTEGER以及FLOAT等枚举实例。这就消除了每个SQL类型都需要一个**@interface**定义的负担，不过也使得以额外的信息修饰SQL类型的需求变得不可能，而这些额外的信息，例如长度或精度等，可能是非常有必要的需求。

我们也可以使用**String**元素来描述实际的SQL类型，比如VARCHAR(30)或INTEGER。这使得程序员可以修饰SQL类型。但是，它同时也将Java类型到SQL类型的映射绑在了一起，这可不是一个好的设计。我们可不希望更换数据库导致代码必须修改并重新编译。如果我们只需告诉注解处理器，我们正在使用的是什么“口味”的SQL，然后由处理器为我们处理SQL类型的细节，那将是一个优雅的设计。

第三种可行的方案是同时使用两个注解类型来注解一个域，**@Constraints**和相应的SQL类型（例如**@SQLIntege**）。这种方式可能会使代码有点乱，不过编译器允许程序员对一个目标同时使用多个注解。注意，使用多个注解的时候，同一个注解不能重复使用。

20.2.4 注解不支持继承

不能使用关键字**extends**来继承某个**@interface**。这真是一个遗憾。如果可以定义一个**@TableColumn**注解（参考前面的建议），同时在其中嵌套一个**@SQLType**类型的注解，那么这将成为一个优雅的设计。按照这种方式，程序员可以继承**@SQLType**，从而创建出各种SQL类型，例如**@SQLInteger**和**@SQLString**等。如果注解允许继承的话，这将大大减少打字的工作量，并且使语法更整洁。在Java未来的版本中，似乎没有任何关于让注解支持继承的提案，所以，在当前状况下，上例中的解决方案可能已经是最佳方法了。

1070

20.2.5 实现处理器

下面是一个注解处理器的例子，它将读取一个类文件，检查其上的数据库注解，并生成用来创建数据库的SQL命令：

```
//: annotations/database/TableCreator.java
// Reflection-based annotation processor.
// {Args: annotations.database.Member}
package annotations.database;
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class TableCreator {
  public static void main(String[] args) throws Exception {
    if(args.length < 1) {
      System.out.println("arguments: annotated classes");
      System.exit(0);
    }
    for(String className : args) {
      Class<?> cl = Class.forName(className);
      DBTable dbTable = cl.getAnnotation(DBTable.class);
      if(dbTable == null) {
        System.out.println(
          "No DBTable annotations in class " + className);
        continue;
      }
      String tableName = dbTable.name();
      // If the name is empty, use the Class name:
      if(tableName.length() < 1)
        tableName = cl.getName().toUpperCase();
      List<String> columnDefs = new ArrayList<String>();
      for(Field field : cl.getDeclaredFields()) {
        String columnName = null;
        Annotation[] anns = field.getDeclaredAnnotations();
        if(anns.length < 1)
          continue; // Not a db table column
        if(anns[0] instanceof SQLInteger) {
          SQLInteger sInt = (SQLInteger) anns[0];
          // Use field name if name not specified
          if(sInt.name().length() < 1)
            columnName = field.getName().toUpperCase();
          else
            columnName = sInt.name();
          columnDefs.add(columnName + " INT" +
            getConstraints(sInt.constraints()));
        }
        if(anns[0] instanceof SQLString) {
          SQLString sString = (SQLString) anns[0];
          // Use field name if name not specified.
          if(sString.name().length() < 1)
            columnName = field.getName().toUpperCase();
          else
            columnName = sString.name();
          columnDefs.add(columnName + " VARCHAR(" +
```

1071

```
            sString.value() + ")" +
            getConstraints(sString.constraints()));
        }
        StringBuilder createCommand = new StringBuilder(
          "CREATE TABLE " + tableName + "(");
        for(String columnDef : columnDefs)
          createCommand.append("\n    " + columnDef + ",");
        // Remove trailing comma
        String tableCreate = createCommand.substring(
          0, createCommand.length() - 1) + ");";
        System.out.println("Table Creation SQL for " +
          className + " is :\n" + tableCreate);
      }
    }
  }
  private static String getConstraints(Constraints con) {
    String constraints = "";
    if(!con.allowNull())
      constraints += " NOT NULL";
    if(con.primaryKey())
      constraints += " PRIMARY KEY";
    if(con.unique())
      constraints += " UNIQUE";
    return constraints;
  }
} /* Output:
Table Creation SQL for annotations.database.Member is :
CREATE TABLE MEMBER(
    FIRSTNAME VARCHAR(30));
Table Creation SQL for annotations.database.Member is :
CREATE TABLE MEMBER(
    FIRSTNAME VARCHAR(30),
    LASTNAME VARCHAR(50));
Table Creation SQL for annotations.database.Member is :
CREATE TABLE MEMBER(
    FIRSTNAME VARCHAR(30),
    LASTNAME VARCHAR(50),
    AGE INT);
Table Creation SQL for annotations.database.Member is :
CREATE TABLE MEMBER(
    FIRSTNAME VARCHAR(30),
    LASTNAME VARCHAR(50),
    AGE INT,
    HANDLE VARCHAR(30) PRIMARY KEY);
*///:~
```

1072

main()方法会处理命令行传入的每一个类名。使用**forName()**方法加载每一个类，并使用**getAnnotation(DBTable.class)**检查该类是否带有**@DBTable**注解。如果有，就将发现的表名保存下来。然后读取这个类的所有域，并用**getDeclaredAnnotation()**进行检查。该方法返回一个包含一个域上的所有注解的数组。最后用**instanceof**操作符来判断这些注解是否是**@SQLIntege**或**@SQLString**类型，如果是的话，在对应的处理块中将构造出相应**cloumn**名的字符串片断。注意，由于注解没有继承机制，所以要获得近似多态的行为，使用**getDeclaredAnnotation()**是唯一的办法。

嵌套中的**@Constraint**注解被传递给**getConstraints()**方法，由它负责构造一个包含SQL约束的**String**对象。

需要提醒读者的是，上面演示的技巧对于真实的对象/关系映射而言，是很幼稚的。例如使用**@DBTable**类型的注解，程序员以参数的形式给出表的名字，如果程序员想要修改表的名字，这将迫使其必须重新编译Java代码。这可不是我们希望看到的结果。现在已经有了很多可用的framework，可以将对象映射到关系数据库，并且，其中越来越多的framework已经开始利用注解了。

练习1：(2) 为本节数据库的例子实现更多的SQL类型。 1073

作业[⊖]：修改数据库的例子，使其能够使用JDBC连接到一个真正的数据库，并与之交互。

作业：修改数据库的例子，令其生成XML构造文件，而不是SQL语句。

20.3 使用apt处理注解

注解处理工具**apt**，这是Sun为了帮助注解的处理过程而提供的工具。由于这是该工具的第一版，其功能还比较基础，不过它确实有助于程序员的开发工作。

与**javac**一样，**apt**被设计为操作Java源文件，而不是编译后的类。默认情况下，**apt**会在处理完源文件后编译它们。如果在系统构建的过程中会自动创建一些新的源文件，那么这个特性非常有用。事实上，**apt**会检查新生成的源文件中注解，然后将所有文件一同编译。

当注解处理器生成一个新的源文件时，该文件会在新一轮（round，Sun文档中这样称呼它）的注解处理中接受检查。该工具会一轮一轮地处理，直到不再有新的源文件产生为止。然后它再编译所有的源文件。

程序员自定义的每一个注解都需要自己的处理器，而**apt**工具能够很容易地将多个注解处理器组合在一起。有了它，程序员就可以指定多个要处理的类，这比程序员自己遍历所有的类文件简单多了。此外还可以添加监听器，并在一轮注解处理过程结束的时候收到通知信息。

在撰写本章的时候，**apt**还不是一个正式的Ant任务（参见http://MindView.net/Books/BetterJava中的附件），不过显然可以将其作为一个Ant的外部任务运行。要想编译这一节中出现的注解处理器，你必须将**tools.jar**设置在你的classpath中，这个工具类库同时还包含了**com.sun.mirror.***接口。

通过使用**AnnotationProcessorFactory**，**apt**能够为每一个它发现的注解生成一个正确的注解处理器。当你使用**apt**的时候，必须指明一个工厂类，或者指明能找到**apt**所需的工厂类的路径。否则，**apt**会踏上一个神秘的探索之旅，详细的信息可以在Sun文档的“开发一个注解处理器”一节中找到。 1074

使用**apt**生成注解处理器时，我们无法利用Java的反射机制，因为我们操作的是源代码，而不是编译后的类[⊜]。使用**mirror API**[⊜]能够解决这个问题，它使我们能够在未经编译的源代码中查看方法，域以及类型。

下面是一个自定义的注解，使用它可以把一个类中的**public**方法提取出来，构造成一个新的接口：

```
//: annotations/ExtractInterface.java
// APT-based annotation processing.
package annotations;
import java.lang.annotation.*;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.SOURCE)
public @interface ExtractInterface {
  public String value();
} ///:~
```

RetentionPolicy是**SOURCE**，因为当我们从一个使用了该注解的类中抽取出接口之后，没有必要再保留这些注解信息。下面的类有一个公共方法，我们将会把它抽取到一个有用接口中：

⊖ 作业，我建议读者将其作为课程大作业。解答指南中不包含此类作业的解决方案。

⊜ 不过，使用非标准的选项**-XclassesAsDecls**，你可以在编译后的类中操作注解。

⊜ Java设计师们卖弄地认为，镜子（mirror）就是起反射（reflection）的作用。

```
//: annotations/Multiplier.java
// APT-based annotation processing.
package annotations;

@ExtractInterface("IMultiplier")
public class Multiplier {
  public int multiply(int x, int y) {
    int total = 0;
    for(int i = 0; i < x; i++)
      total = add(total, y);
    return total;
  }
  private int add(int x, int y) { return x + y; }
  public static void main(String[] args) {
    Multiplier m = new Multiplier();
    System.out.println("11*16 = " + m.multiply(11, 16));
  }
} /* Output:
11*16 = 176
*///:~
```

1075

在**Multiplier**类中（它只对正整数起作用）有一个**multiply()**方法，该方法多次调用一个私有的**add()**方法以实现乘法操作。**add()**方法不是公共的，因此不将其作为接口的一部分。注解给出了值**IMultiplier**，这就是将要生成的接口的名字：

```
//: annotations/InterfaceExtractorProcessor.java
// APT-based annotation processing.
// {Exec: apt -factory
// annotations.InterfaceExtractorProcessorFactory
// Multiplier.java -s ../annotations}
package annotations;
import com.sun.mirror.apt.*;
import com.sun.mirror.declaration.*;
import java.io.*;
import java.util.*;

public class InterfaceExtractorProcessor
  implements AnnotationProcessor {
  private final AnnotationProcessorEnvironment env;
  private ArrayList<MethodDeclaration> interfaceMethods =
    new ArrayList<MethodDeclaration>();
  public InterfaceExtractorProcessor(
    AnnotationProcessorEnvironment env) { this.env = env; }
  public void process() {
    for(TypeDeclaration typeDecl :
      env.getSpecifiedTypeDeclarations()) {
      ExtractInterface annot =
        typeDecl.getAnnotation(ExtractInterface.class);
      if(annot == null)
        break;
      for(MethodDeclaration m : typeDecl.getMethods())
        if(m.getModifiers().contains(Modifier.PUBLIC) &&
           !(m.getModifiers().contains(Modifier.STATIC)))
          interfaceMethods.add(m);
      if(interfaceMethods.size() > 0) {
        try {
          PrintWriter writer =
            env.getFiler().createSourceFile(annot.value());
          writer.println("package " +
            typeDecl.getPackage().getQualifiedName() +";");
          writer.println("public interface " +
            annot.value() + " {");
          for(MethodDeclaration m : interfaceMethods) {
            writer.print("  public ");
            writer.print(m.getReturnType() + " ");
            writer.print(m.getSimpleName() + " (");
```

1076

```
            int i = 0;
            for(ParameterDeclaration parm :
              m.getParameters()) {
              writer.print(parm.getType() + " " +
                parm.getSimpleName());
              if(++i < m.getParameters().size())
                writer.print(", ");
            }
            writer.println(");");
          }
          writer.println("}");
          writer.close();
        } catch(IOException ioe) {
          throw new RuntimeException(ioe);
        }
      }
    }
  }
} ///:~
```

所有的工作都在**process()**方法中完成。在分析一个类的时候，我们用**MethodDeclaration**类以及其上的**getModifiers()**方法来找到**public**方法（不包括**static**的那些）。一旦找到我们所需的**public**方法，就将其保存在一个**ArrayList**中，然后在一个**.java**文件中，创建新的接口中的方法定义。

注意，处理器类的构造器以**AnnotationProcessorEnvironment**对象为参数。通过该对象，我们就能知道**apt**正在处理的所有类型（类定义），并且可以通过它获得**Messager**对象和**Filer**对象。**Messager**对象可以用来向用户报告信息，比如处理过程中发生的任何错误，以及错误在源代码中出现的位置等。**Filer**是一种**PrintWriter**，我们可以通过它创建新的文件。不使用普通的**PrintWriter**而使用**Filer**对象的主要原因是，只有这样**apt**才能知道我们创建的新文件，从而对新文件进行注解处理，并且在需要的时候编译它们。

1077

同时我们看到，**Filer**的**createSourceFile()**方法以将要新建的类或接口的名字，打开了一个普通的输出流。现在还没有什么工具帮助程序员创建Java语言结构，所以我们只能用基本的**print()**和**println()**方法来生成Java源代码。因此，你必须小心仔细地处理括号，确保其闭合，并且确保生成的代码语法正确。

apt工具需要一个工厂类来为其指明正确的处理器，然后它才能调用处理器上的**process()**方法：

```
//: annotations/InterfaceExtractorProcessorFactory.java
// APT-based annotation processing.
package annotations;
import com.sun.mirror.apt.*;
import com.sun.mirror.declaration.*;
import java.util.*;

public class InterfaceExtractorProcessorFactory
  implements AnnotationProcessorFactory {
  public AnnotationProcessor getProcessorFor(
    Set<AnnotationTypeDeclaration> atds,
    AnnotationProcessorEnvironment env) {
    return new InterfaceExtractorProcessor(env);
  }
  public Collection<String> supportedAnnotationTypes() {
    return
     Collections.singleton("annotations.ExtractInterface");
  }
  public Collection<String> supportedOptions() {
    return Collections.emptySet();
  }
} ///:~
```

1078

AnnotationProcessorFactory接口只有三个方法。如你所见，其中之一的**getProcessorFor()**方法返回注解处理器，该方法以包含类型声明的**Set**（使用**apt**工具时传入的Java类）以及**AnnotationProcessorEnvironment**对象为参数（将传入给处理器对象）。另外两个方法是**supportedAnnotationTypes()**和**supportedOptions()**，程序员可以通过它们检查一下，是否**apt**工具发现的所有的注解都有相应的处理器，是否所有控制台输入的参数都是你提供支持的选项。其中，**supportedAnnotationTypes ()**方法尤其重要，因为一旦在返回的**String**集合中没有你的注解的完整类名，**apt**就会抱怨没有找到相应的处理器，从而发出警告信息，然后什么也不做就退出。

以上例子中的处理器与工厂类都在**annotations**包中，在**InterfaceExtractorProcessor.java**开头的注释文字中，我根据**anotations**的目录结构，在**Exec**标记处给出了需要从命令行输入的命令。它将告诉**apt**工具，使用上面的工厂类来处理**Multiplier.java**文件。参数**-s**说明任何新产生的文件都必须放在**annotations**目录中。通过处理器中的**println()**语句，估计你已经能猜到最终生成的**IMultiplier.java**会是什么样子了：

```
package annotations;
public interface IMultiplier {
 public int multiply (int x, int y);
}
```

apt也会编译这个新产生的文件，因此你将在相同的目录中看到**IMultiplier.class**文件。

练习2：(3) 为抽取出来的接口添加对除法的支持。

20.4 将观察者模式用于apt

上面的例子是一个相当简单的注解处理器，只需对一个注解进行分析，但我们仍然要做大量复杂的工作。因此，处理注解的真实过程可能会非常复杂。当我们有更多的注解和更多的处理器时，为了防止这种复杂性迅速攀升，**mirror** API提供了对访问者设计模式的支持。访问者是Gamma等人所著的《设计模式》[⊖]一书中的经典设计模式之一。你也可以在《Thinking in Patterns》中找到更详细的解释。

1079

一个访问者会遍历某个数据结构或一个对象的集合，对其中的每一个对象执行一个操作。该数据结构无需有序，而你对每个对象执行的操作，都是特定于此对象的类型。这就将操作与对象解耦，也就是说，你可以添加新的操作，而无需向类的定义中添加方法。

这个技巧在处理注解时非常有用，因为一个Java类可以看作是一系列对象的集合，例如**TypeDeclaration**对象、**FieldDeclaration**对象以及**MethodDeclaration**对象等。当你配合访问者模式使用**apt**工具时，需要提供一个**Visitor**类，它具有一个能够处理你要访问的各种声明的方法。然后，你就可以为方法、类以及域上的注解实现相应的处理行为。

下面仍然是SQL表生成器的例子，不过这次我们使用访问者模式来创建工厂和注解处理器：

```
//: annotations/database/TableCreationProcessorFactory.java
// The database example using Visitor.
// {Exec: apt -factory
// annotations.database.TableCreationProcessorFactory
// database/Member.java -s database}
package annotations.database;
import com.sun.mirror.apt.*;
import com.sun.mirror.declaration.*;
import com.sun.mirror.util.*;
import java.util.*;
import static com.sun.mirror.util.DeclarationVisitors.*;
```

⊖ 本书中文版、英文版以及双语版均已由机械工业出版社出版——编辑注。

```
public class TableCreationProcessorFactory
  implements AnnotationProcessorFactory {
  public AnnotationProcessor getProcessorFor(
    Set<AnnotationTypeDeclaration> atds,
    AnnotationProcessorEnvironment env) {
    return new TableCreationProcessor(env);
  }
  public Collection<String> supportedAnnotationTypes() {
    return Arrays.asList(
      "annotations.database.DBTable",
      "annotations.database.Constraints",
      "annotations.database.SQLString",
      "annotations.database.SQLInteger");
  }
  public Collection<String> supportedOptions() {
    return Collections.emptySet();
  }
  private static class TableCreationProcessor
    implements AnnotationProcessor {
    private final AnnotationProcessorEnvironment env;
    private String sql = "";
    public TableCreationProcessor(
      AnnotationProcessorEnvironment env) {
      this.env = env;
    }
    public void process() {
      for(TypeDeclaration typeDecl :
        env.getSpecifiedTypeDeclarations()) {
        typeDecl.accept(getDeclarationScanner(
          new TableCreationVisitor(), NO_OP));
        sql = sql.substring(0, sql.length() - 1) + ");";
        System.out.println("creation SQL is :\n" + sql);
        sql = "";
      }
    }
    private class TableCreationVisitor
      extends SimpleDeclarationVisitor {
      public void visitClassDeclaration(
        ClassDeclaration d) {
        DBTable dbTable = d.getAnnotation(DBTable.class);
        if(dbTable != null) {
          sql += "CREATE TABLE ";
          sql += (dbTable.name().length() < 1)
            ? d.getSimpleName().toUpperCase()
            : dbTable.name();
          sql += " (";
        }
      }
      public void visitFieldDeclaration(
        FieldDeclaration d) {
        String columnName = "";
        if(d.getAnnotation(SQLInteger.class) != null) {
          SQLInteger sInt = d.getAnnotation(
              SQLInteger.class);
          // Use field name if name not specified
          if(sInt.name().length() < 1)
            columnName = d.getSimpleName().toUpperCase();
          else
            columnName = sInt.name();
          sql += "\n    " + columnName + " INT" +
            getConstraints(sInt.constraints()) + ",";
        }
        if(d.getAnnotation(SQLString.class) != null) {
          SQLString sString = d.getAnnotation(
              SQLString.class);
          // Use field name if name not specified.
          if(sString.name().length() < 1)
```

1080

1081

```
            columnName = d.getSimpleName().toUpperCase();
          else
            columnName = sString.name();
          sql += "\n    " + columnName + " VARCHAR(" +
            sString.value() + ")" +
            getConstraints(sString.constraints()) + ",";
        }
      }
      private String getConstraints(Constraints con) {
        String constraints = "";
        if(!con.allowNull())
          constraints += " NOT NULL";
        if(con.primaryKey())
          constraints += " PRIMARY KEY";
        if(con.unique())
          constraints += " UNIQUE";
        return constraints;
      }
    }
  }
} ///:~
```

这个程序输出的结果与前一个**DBTable**的例子完全相同。

在这个例子中，处理器与访问者都是内部类。注意，**process()**方法所做的只是添加了一个访问者类，并初始化了SQL字符串。

getDeclarationScanner()方法的两个参数都是访问者：第一个是在访问每个声明前使用，第二个则是在访问之后使用。由于这个处理器只需要在访问前使用的访问者，所以第二个参数给的是**NO_OP**。**NO_OP**是**DeclarationVisitor**接口中的**static**域，是一个什么也不做的**Declaration-Visitor**。

1082

TableCreationVisitor继承自**SimpleDeclarationVisitor**，它覆写了两个方法**visitClaseDeclaration()** 和**visitFieldDeclaration()**。**SimpleDeclarationVisitor**是一个适配器，实现了**DeclarationVisitor**接口中的所有方法，因此，程序员只需将注意力放在自己需要的那些方法上。在**visitClaseDeclaration()**方法中，检查**ClassDeclaration**对象是否带有**DBTable**注解，如果存在的话，将初始化SQL语句的第一部分。在**visitFieldDeclaration()**方法中，将检查域声明上的注解，从域声明中提取信息的过程与本章前面的例子一样。

看起来这个例子使用的方式似乎更复杂，但是它确实是一种具备扩展能力的解决方案。当你的注解处理器的复杂性越来越高的时候，如果还按前面例子中的方式编写自己独立的处理器，那么很快你的处理器就将变得非常复杂。

练习3：(2) 向**TableCreationProcessorFactory.java**中添加对更多的SQL类型的支持。

20.5 基于注解的单元测试

*单元测试*是对类中的每个方法提供一个或多个测试的一种实践，其目的是为了有规律地测试一个类的各个部分是否具备正确的行为。在Java中，最著名的单元测试工具就是JUnit。在撰写本书时，JUnit已经开始了向JUnit4更新的过程，其目的正是为了融入注解[⊖]。对于注解出现之前的JUnit而言，有一个主要的问题，即为了设置并运行JUnit测试需要做大量的形式上的工作。随着其渐渐的发展，这种负担已经减轻了一些，但注解的出现能够使其更贴近“最简单的单元测试系统”。

使用注解出现之前的JUnit，程序员必须创建一个独立的类来保存其单元测试。有了注解，

⊖ 我原本考虑过基于这里的设计来自己做一个Better JUnit。不过后来发现JUnit4已经具有很多我这里讲到的思想，因此直接升级为JUnit4可能更简单吧。

我们可以直接在要验证的类里面编写测试，这将大大减少单元测试所需的时间和麻烦之处。采用这种方式还有一个额外的好处，就是能够像测试**public**方法一样很容易地测试**private**方法。 1083

这个基于注解的测试框架叫做**@Unit**。其最基本的测试形式，可能也是你用的最多的一个注解是**@Test**，我们用**@Test**来标记测试方法。测试方法不带参数，并返回**boolean**结果来说明测试成功或失败。程序员可以任意命名他的测试方法。同时，**@Unit**测试方法可以是任意你喜欢的访问修饰方式，包括**private**。

要使用**@Unit**，程序员必须引入**net.mindview.atunit**[⊖]，用**@Unit**的测试标记为合适的方法和域打上标记（在接下来的例子中你会学到），然后让你的构建系统对编译后的类运行**@Unit**。下面是一个简单的例子：

```
//: annotations/AtUnitExample1.java
package annotations;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitExample1 {
  public String methodOne() {
    return "This is methodOne";
  }
  public int methodTwo() {
    System.out.println("This is methodTwo");
    return 2;
  }
  @Test boolean methodOneTest() {
    return methodOne().equals("This is methodOne");
  }
  @Test boolean m2() { return methodTwo() == 2; }
  @Test private boolean m3() { return true; }
  // Shows output for failure:
  @Test boolean failureTest() { return false; }
  @Test boolean anotherDisappointment() { return false; }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit AtUnitExample1");
  }
} /* Output:
annotations.AtUnitExample1
  . methodOneTest
  . m2 This is methodTwo

  . m3
  . failureTest (failed)
  . anotherDisappointment (failed)
(5 tests)

>>> 2 FAILURES <<<
  annotations.AtUnitExample1: failureTest
  annotations.AtUnitExample1: anotherDisappointment
*///:~
```

1084

使用**@Unit**进行测试的类必须定义在某个包中（即必须包括**packae**声明）。

@Test注解被置于**methodOneTest()**、**m2()**、**m3()failureTest()**以及**anotherDisappointment()**方法之前，它告诉**@Unit**将这些方法作为单元测试来运行。同时，**@Test**将验证并确保这些方法没有参数，并且返回值是**boolean**或**void**。程序员编写单元测试时，唯一需要做的就是决定测试是成功还是失败，（对于返回值为**boolean**的方法）应该返回**ture**还是**false**。

如果你熟悉JUnit，你会注意到**@Unit**的输出带有更多的信息。我们可以看到当前正在运行

⊖ 这个类库是本书附带的代码包的一部分，可以在www.MindView.net上找到。

的测试，因此测试中的输出更是有用，而且在最后，它还能告诉我们导致错误的类和测试。

程序员并非必须将测试方法嵌入到原本的类中，因为有时候这根本做不到。要生成一个非嵌入式的测试，最简单的办法就是继承：

```
//: annotations/AtUnitExternalTest.java
// Creating non-embedded tests.
package annotations;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitExternalTest extends AtUnitExample1 {
  @Test boolean _methodOne() {
    return methodOne().equals("This is methodOne");
  }
  @Test boolean _methodTwo() { return methodTwo() == 2; }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
     "java net.mindview.atunit.AtUnit AtUnitExternalTest");
  }
} /* Output:
annotations.AtUnitExternalTest
  . _methodOne
  . _methodTwo This is methodTwo

OK (2 tests)
*///:~
```

1085

这个例子还表现出了灵活命名的价值（与JUnit不同，它要求你必须使用**test**作为测试方法的前缀）。在这里，**@Test**方法被命名为下划线前缀加上这将要测试的方法的名字（我并不认为这是一个理想的命名形式，只是表现一种可能性罢了）。

或者你还可以使用组合的方式创建非嵌入式的测试：

```
//: annotations/AtUnitComposition.java
// Creating non-embedded tests.
package annotations;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitComposition {
  AtUnitExample1 testObject = new AtUnitExample1();
  @Test boolean _methodOne() {
    return
      testObject.methodOne().equals("This is methodOne");
  }
  @Test boolean _methodTwo() {
    return testObject.methodTwo() == 2;
  }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
     "java net.mindview.atunit.AtUnit AtUnitComposition");
  }
} /* Output:
annotations.AtUnitComposition
  . _methodOne
  . _methodTwo This is methodTwo

OK (2 tests)
*///:~
```

1086

因为每个测试对应一个新创建的**AtUnitComposition**对象，因此每个测试也对应一个新的成员**testObject**。

@Unit中并没有JUnit里的特殊的**assert**方法，不过**@Test**方法仍然允许程序员返回**void**（如果你还是想用**ture**或**false**的话，你仍然可以用**boolean**作为方法返回值类型），这是**@Test**方法的

第二种形式。在这种情况下，要表示测试成功，可以使用Java的**assert**语句。Java的断言机制一般要求程序员在java命令行中加上**-ea**标志，不过**@Unit**已经自动打开了该功能。而要表示测试失败的话，你甚至可以使用异常。**@Unit**的设计目标之一就是尽可能少地添加额外的语法，而Java的**assert**和异常对于报告错误而言，已经足够了。一个失败的**assert**或从测试方法中抛出异常，都将被看作一个失败的测试，但是**@Unit**并不会就在这个失败的测试上打住，它会继续运行，直到所有的测试都运行完毕。下面是一个示例程序：

```
//: annotations/AtUnitExample2.java
// Assertions and exceptions can be used in @Tests.
package annotations;
import java.io.*;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitExample2 {
  public String methodOne() {
    return "This is methodOne";
  }
  public int methodTwo() {
    System.out.println("This is methodTwo");
    return 2;
  }
  @Test void assertExample() {
    assert methodOne().equals("This is methodOne");
  }
  @Test void assertFailureExample() {
    assert 1 == 2: "What a surprise!";
  }
  @Test void exceptionExample() throws IOException {
    new FileInputStream("nofile.txt"); // Throws
  }
  @Test boolean assertAndReturn() {
    // Assertion with message:
    assert methodTwo() == 2: "methodTwo must equal 2";
    return methodOne().equals("This is methodOne");
  }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit AtUnitExample2");
  }
} /* Output:
annotations.AtUnitExample2
  . assertExample
  . assertFailureExample java.lang.AssertionError: What a
surprise!
(failed)
  . exceptionExample java.io.FileNotFoundException:
nofile.txt (The system cannot find the file specified)
(failed)
  . assertAndReturn This is methodTwo

(4 tests)

>>> 2 FAILURES <<<
  annotations.AtUnitExample2: assertFailureExample
  annotations.AtUnitExample2: exceptionExample
*///:~
```

下面的例子使用非嵌入式的测试，并且用到了断言，它将对**java.util.HashSet**执行一些简单的测试：

```
//: annotations/HashSetTest.java
package annotations;
```

```
import java.util.*;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class HashSetTest {
  HashSet<String> testObject = new HashSet<String>();
  @Test void initialization() {
    assert testObject.isEmpty();
  }
  @Test void _contains() {
    testObject.add("one");
    assert testObject.contains("one");
  }
  @Test void _remove() {
    testObject.add("one");
    testObject.remove("one");
    assert testObject.isEmpty();
  }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit HashSetTest");
  }
} /* Output:
annotations.HashSetTest
  . initialization
  . _remove
  . _contains
OK (3 tests)
*///:~
```

1088

如果采用继承的方式，可能会更简单，并且也没有一些其他的约束。

练习4：(3) 验证是否每个测试都会生成一个新的**testObject**。

练习5：(1) 使用继承的方式修改上面的例子。

练习6：(1) 使用**HashSetTest.java**演示的方式测试**LinkedList**类。

练习7：(1) 使用继承的方式修改前一个练习的结果。

对每一个单元测试而言，**@Unit**都会用默认的构造器，为该测试所属的类创建出一个新的实例。并在此新创建的对象上运行测试，然后丢弃该对象，以避免对其他测试产生副作用。如此创建对象导致我们依赖于类的默认构造器。如果你的类没有默认构造器，或者新对象需要复杂的构造过程，那么你可以创建一个**static**方法专门负责构造对象，然后用**@TestObjectCreaet**注解将该方法标记出来，就像这样：

```
//: annotations/AtUnitExample3.java
package annotations;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitExample3 {
  private int n;
  public AtUnitExample3(int n) { this.n = n; }
  public int getN() { return n; }
  public String methodOne() {
    return "This is methodOne";
  }
  public int methodTwo() {
    System.out.println("This is methodTwo");
    return 2;
  }
  @TestObjectCreate static AtUnitExample3 create() {
    return new AtUnitExample3(47);
  }
  @Test boolean initialization() { return n == 47; }
```

1089

```
  @Test boolean methodOneTest() {
    return methodOne().equals("This is methodOne");
  }
  @Test boolean m2() { return methodTwo() == 2; }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit AtUnitExample3");
  }
} /* Output:
annotations.AtUnitExample3
  . initialization
  . methodOneTest
  . m2 This is methodTwo

OK (3 tests)
*///:~
```

加入了**@TestObjectCreaet**注解的方法必须声明为**static**，且必须返回一个你正在测试的类型的对象，这一切都由**@Unit**负责确保成立。

有的时候，我们需要向单元测试中添加一些额外的域。这时可以使用**@TestProperty**注解，由它注解的域表示只在单元测试中使用（因此，在我们将产品发布给客户之前，他们应该被删除掉）。在下面的例子中，一个**String**通过**String.split()**方法被拆散了，从其中读取一个值，这个值将被用来生成测试对象：

```
//: annotations/AtUnitExample4.java
package annotations;
import java.util.*;
import net.mindview.atunit.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;
public class AtUnitExample4 {
  static String theory = "All brontosauruses " +
    "are thin at one end, much MUCH thicker in the " +
    "middle, and then thin again at the far end.";
  private String word;
  private Random rand = new Random(); // Time-based seed
  public AtUnitExample4(String word) { this.word = word; }
  public String getWord() { return word; }
  public String scrambleWord() {
    List<Character> chars = new ArrayList<Character>();
    for(Character c : word.toCharArray())
      chars.add(c);
    Collections.shuffle(chars, rand);
    StringBuilder result = new StringBuilder();
    for(char ch : chars)
      result.append(ch);
    return result.toString();
  }
  @TestProperty static List<String> input =
    Arrays.asList(theory.split(" "));
  @TestProperty
    static Iterator<String> words = input.iterator();
  @TestObjectCreate static AtUnitExample4 create() {
    if(words.hasNext())
      return new AtUnitExample4(words.next());
    else
      return null;
  }
  @Test boolean words() {
    print("'" + getWord() + "'");
    return getWord().equals("are");
  }
  @Test boolean scramble1() {
    // Change to a specific seed to get verifiable results:
```

1090

```
    rand = new Random(47);
    print("'" + getWord() + "'");
    String scrambled = scrambleWord();
    print(scrambled);
    return scrambled.equals("lAl");
  }
  @Test boolean scramble2() {
    rand = new Random(74);
    print("'" + getWord() + "'");
    String scrambled = scrambleWord();
    print(scrambled);
    return scrambled.equals("tsaeborornussu");
  }
  public static void main(String[] args) throws Exception {
    System.out.println("starting");
    OSExecute.command(
      "java net.mindview.atunit.AtUnit AtUnitExample4");
  }
} /* Output:
starting
annotations.AtUnitExample4
  . scramble1 'All'
lAl

  . scramble2 'brontosauruses'
tsaeborornussu

  . words 'are'

OK (3 tests)
*///:~
```

1091

@TestProperty也可以用来标记那些只在测试中使用的方法，而他们本身又不是测试方法。注意，这个程序依赖于测试执行的顺序，这可不是一个好的实践。

如果你的测试对象需要执行某些初始化工作，并且使用完毕后还需要进行某些清理工作，那么可以选择使用**static @TestObjectCleanup**方法，当测试对象使用结束后，该方法会为你执行清理工作。在下面的例子中，**@TestObjectCreate**为每个测试对象打开了一个文件，因此必须在丢弃测试对象的时候关闭该文件：

```
//: annotations/AtUnitExample5.java
package annotations;
import java.io.*;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class AtUnitExample5 {
  private String text;
  public AtUnitExample5(String text) { this.text = text; }
  public String toString() { return text; }
  @TestProperty static PrintWriter output;
  @TestProperty static int counter;
  @TestObjectCreate static AtUnitExample5 create() {
    String id = Integer.toString(counter++);
    try {
      output = new PrintWriter("Test" + id + ".txt");
    } catch(IOException e) {
      throw new RuntimeException(e);
    }
    return new AtUnitExample5(id);
  }
  @TestObjectCleanup static void
  cleanup(AtUnitExample5 tobj) {
    System.out.println("Running cleanup");
    output.close();
```

1092

```
  }
  @Test boolean test1() {
    output.print("test1");
    return true;
  }
  @Test boolean test2() {
    output.print("test2");
    return true;
  }
  @Test boolean test3() {
    output.print("test3");
    return true;
  }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit AtUnitExample5");
  }
} /* Output:
annotations.AtUnitExample5
  . test1
Running cleanup
  . test2
Running cleanup
  . test3
Running cleanup
OK (3 tests)
*///:~
```

从输出中我们可以看到，清理方法会在每个测试结束后自动运行。 1093

20.5.1 将@Unit用于泛型

泛型为**@Unit**出了一个难题，因为我们不可能“泛泛地测试”。我们必须针对某个特定类型的参数或参数集才能进行测试。解决的办法很简单：让测试类继承自泛型类的一个特定版本即可。

下面是一个堆栈的例子：

```
//: annotations/StackL.java
// A stack built on a linkedList.
package annotations;
import java.util.*;

public class StackL<T> {
  private LinkedList<T> list = new LinkedList<T>();
  public void push(T v) { list.addFirst(v); }
  public T top() { return list.getFirst(); }
  public T pop() { return list.removeFirst(); }
} ///:~
```

要测试**String**版的堆栈，就让测试类继承自**StackL<String>**：

```
//: annotations/StackLStringTest.java
// Applying @Unit to generics.
package annotations;
import net.mindview.atunit.*;
import net.mindview.util.*;

public class StackLStringTest extends StackL<String> {
  @Test void _push() {
    push("one");
    assert top().equals("one");
    push("two");
    assert top().equals("two");
  }
  @Test void _pop() {
    push("one");
    push("two");
    assert pop().equals("two");
```

```
    assert pop().equals("one");
  }
  @Test void _top() {
    push("A");
    push("B");
    assert top().equals("B");
    assert top().equals("B");
  }
  public static void main(String[] args) throws Exception {
    OSExecute.command(
      "java net.mindview.atunit.AtUnit StackLStringTest");
  }
} /* Output:
annotations.StackLStringTest
  . _push
  . _pop
  . _top
OK (3 tests)
*///:~
```

1094

这种方法潜在的唯一缺点是：继承使我们失去了访问被测试的类中的**private**方法的能力。如果这对你很重要，那你要么将**private**方法改为**protected**，要么添加一个非**private**的**@TestProperty**方法，由它来调用**private**方法（稍候我们会看到，**AtUnitRemover**工具会将**@TestProperty**方法从产品的代码中自动删除掉）。

练习8：(2) 写一个带有**private**方法的类，然后像上介绍的那样添加一个非**private @TestProperty**方法，并在你的测试代码中调用此方法。

练习9：(2) 为**HashMap**编写一些基本的**@Unit**测试。

练习10：(2) 从本书中选择一个示例程序，为其编写**@Unit**测试。

20.5.2 不需要任何“套件”

与JUnit相比，**@Unit**有一个比较大的优点，就是**@Unit**不需要“套件”(suites)。在JUnit中，程序员必须告诉测试工具你打算测试什么，这就要求用套件来组织测试，以便JUnit能够找到它们，并运行其中包含的测试。

@Unit只是简单地搜索类文件，检查其是否具有恰当的注解，然后运行**@Test**方法。我的主要目标就是使**@Unit**测试系统尽可能的透明，使得程序员在用它的时候只需添加**@Test**方法，而不需要像JUnit等其他单元测试框架所要求的那些特殊的编码或者知识。不过，如果说编写测试不会遇到任何障碍，这也不太可能，因此**@Unit**会尽量让这些困难变得微不足道。希望通过这种方式，程序员会更乐意编写测试。

1095

20.5.3 实现@Unit

首先，我们需要定义所有的注解类型。这些都是简单的标签，并且没有属性。**@Test**标签在本章开头已经定义过了，这里是其他所需的注解：

```
//: net/mindview/atunit/TestObjectCreate.java
// The @Unit @TestObjectCreate tag.
package net.mindview.atunit;
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TestObjectCreate {} ///:~
```

```
//: net/mindview/atunit/TestObjectCleanup.java
// The @Unit @TestObjectCleanup tag.
package net.mindview.atunit;
import java.lang.annotation.*;

@Target(ElementType.METHOD)
```

```
@Retention(RetentionPolicy.RUNTIME)
public @interface TestObjectCleanup {} ///:~

//: net/mindview/atunit/TestProperty.java
// The @Unit @TestProperty tag.
package net.mindview.atunit;
import java.lang.annotation.*;

// Both fields and methods may be tagged as properties:
@Target({ElementType.FIELD, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
public @interface TestProperty {} ///:~
```

所有测试的保留属性必须是**RUNTIME**，因为**@Unit**系统必须在编译后的代码中查询这些注解。

要实现该系统，并运行测试，我们还需使用反射机制来抽取注解。下面这个程序通过注解中的信息，决定如何构造测试对象，并在测试对象上运行测试。正是由于注解的帮助，这个程序才如此短小而直接：

1096

```
//: net/mindview/atunit/AtUnit.java
// An annotation-based unit-test framework.
// {RunByHand}
package net.mindview.atunit;
import java.lang.reflect.*;
import java.io.*;
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class AtUnit implements ProcessFiles.Strategy {
  static Class<?> testClass;
  static List<String> failedTests= new ArrayList<String>();
  static long testsRun = 0;
  static long failures = 0;
  public static void main(String[] args) throws Exception {
    ClassLoader.getSystemClassLoader()
      .setDefaultAssertionStatus(true); // Enable asserts
    new ProcessFiles(new AtUnit(), "class").start(args);
    if(failures == 0)
      print("OK (" + testsRun + " tests)");
    else {
      print("(" + testsRun + " tests)");
      print("\n>>> " + failures + " FAILURE" +
        (failures > 1 ? "S" : "") + " <<<");
      for(String failed : failedTests)
        print("  " + failed);
    }
  }
  public void process(File cFile) {
    try {
      String cName = ClassNameFinder.thisClass(
        BinaryFile.read(cFile));
      if(!cName.contains("."))
        return; // Ignore unpackaged classes
      testClass = Class.forName(cName);
    } catch(Exception e) {
      throw new RuntimeException(e);
    }
    TestMethods testMethods = new TestMethods();
    Method creator = null;
    Method cleanup = null;
    for(Method m : testClass.getDeclaredMethods()) {
      testMethods.addIfTestMethod(m);
      if(creator == null)
        creator = checkForCreatorMethod(m);
```

1097

```
    if(cleanup == null)
      cleanup = checkForCleanupMethod(m);
  }
  if(testMethods.size() > 0) {
    if(creator == null)
      try {
        if(!Modifier.isPublic(testClass
            .getDeclaredConstructor().getModifiers())) {
          print("Error: " + testClass +
            " default constructor must be public");
          System.exit(1);
        }
      } catch(NoSuchMethodException e) {
        // Synthesized default constructor; OK
      }
    print(testClass.getName());
  }
  for(Method m : testMethods) {
    printnb("  . " + m.getName() + " ");
    try {
      Object testObject = createTestObject(creator);
      boolean success = false;
      try {
        if(m.getReturnType().equals(boolean.class))
          success = (Boolean)m.invoke(testObject);
        else {
          m.invoke(testObject);
          success = true; // If no assert fails
        }
      } catch(InvocationTargetException e) {
        // Actual exception is inside e:
        print(e.getCause());
      }
      print(success ? "" : "(failed)");
      testsRun++;
      if(!success) {
        failures++;
        failedTests.add(testClass.getName() +
          ": " + m.getName());
      }
      if(cleanup != null)
        cleanup.invoke(testObject, testObject);
    } catch(Exception e) {
      throw new RuntimeException(e);
    }
  }
}
static class TestMethods extends ArrayList<Method> {
  void addIfTestMethod(Method m) {
    if(m.getAnnotation(Test.class) == null)
      return;
    if(!(m.getReturnType().equals(boolean.class) ||
        m.getReturnType().equals(void.class)))
      throw new RuntimeException("@Test method" +
        " must return boolean or void");
    m.setAccessible(true); // In case it's private, etc.
    add(m);
  }
}
private static Method checkForCreatorMethod(Method m) {
  if(m.getAnnotation(TestObjectCreate.class) == null)
    return null;
  if(!m.getReturnType().equals(testClass))
    throw new RuntimeException("@TestObjectCreate " +
      "must return instance of Class to be tested");
  if((m.getModifiers() &
      java.lang.reflect.Modifier.STATIC) < 1)
```

1098

```
      throw new RuntimeException("@TestObjectCreate " +
        "must be static.");
    m.setAccessible(true);
    return m;
  }
  private static Method checkForCleanupMethod(Method m) {
    if(m.getAnnotation(TestObjectCleanup.class) == null)
      return null;
    if(!m.getReturnType().equals(void.class))
      throw new RuntimeException("@TestObjectCleanup " +
        "must return void");
    if((m.getModifiers() &
        java.lang.reflect.Modifier.STATIC) < 1)
      throw new RuntimeException("@TestObjectCleanup " +
        "must be static.");
    if(m.getParameterTypes().length == 0 ||
       m.getParameterTypes()[0] != testClass)
      throw new RuntimeException("@TestObjectCleanup " +
        "must take an argument of the tested type.");
    m.setAccessible(true);
    return m;
  }
  private static Object createTestObject(Method creator) {
    if(creator != null) {
      try {
        return creator.invoke(testClass);
      } catch(Exception e) {
        throw new RuntimeException("Couldn't run " +
          "@TestObject (creator) method.");
      }
    } else { // Use the default constructor:
      try {
        return testClass.newInstance();
      } catch(Exception e) {
        throw new RuntimeException("Couldn't create a " +
          "test object. Try using a @TestObject method.");
      }
    }
  }
} ///:~
```

1099

AtUnit.java使用了**net.mindview.util**中的**ProcessFiles**工具。这个类还实现了**ProcessFiles.Strategy**接口，该接口包含**process()**方法。如此一来，便可以将一个**AtUnit**实例传给**ProcessFiles**的构造器。**ProcessFiles**构造器的第二个参数告诉**ProcessFiles**查找所有扩展名为class的文件。

如果你没有提供命令行参数，这个程序会遍历当前目录。你也可以为其提供多个参数，可以是类文件（带有或不带**.class**扩展名都可），或者是一些目录。由于**@Unit**将会自动找到可测试的类和方法，所以没有“套件”机制的必要[⊖]。

AtUnit.java必须要解决一个问题，就是当它找到类文件时，实际引用的类名（含有包）并非一定就是类文件的名字。为了从中解读信息，我们必须分析该类文件，这很重要，因为这种名字不一致的情况确实可能出现[⊜]。所以，当找到一个**.class**文件时，第一件事情就是打开该文件，读取其二进制数据，然后将其交给**ClassNameFinder.thisClass()**。从这里开始，我们将进入“字节码工程”的领域，因为我们实际上是在分析一个类文件的内容：

1100

```
//: net/mindview/atunit/ClassNameFinder.java
package net.mindview.atunit;
```

⊖ 现在还不清楚为何测试所属的类的默认构造器必须是**public**，如果不是的话，调用**newInstantce()**方法会导致程序中止（没有异常抛出）。

⊜ Jeremy Meyer与我在这个问题上花了一整天。

```
import java.io.*;
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;

public class ClassNameFinder {
  public static String thisClass(byte[] classBytes) {
    Map<Integer,Integer> offsetTable =
      new HashMap<Integer,Integer>();
    Map<Integer,String> classNameTable =
      new HashMap<Integer,String>();
    try {
      DataInputStream data = new DataInputStream(
        new ByteArrayInputStream(classBytes));
      int magic = data.readInt();  // 0xcafebabe
      int minorVersion = data.readShort();
      int majorVersion = data.readShort();
      int constant_pool_count = data.readShort();
      int[] constant_pool = new int[constant_pool_count];
      for(int i = 1; i < constant_pool_count; i++) {
        int tag = data.read();
        int tableSize;
        switch(tag) {
          case 1: // UTF
            int length = data.readShort();
            char[] bytes = new char[length];
            for(int k = 0; k < bytes.length; k++)
              bytes[k] = (char)data.read();
            String className = new String(bytes);
            classNameTable.put(i, className);
            break;
          case 5: // LONG
          case 6: // DOUBLE
            data.readLong(); // discard 8 bytes
            i++; // Special skip necessary
            break;
          case 7: // CLASS
            int offset = data.readShort();
            offsetTable.put(i, offset);
            break;
          case 8: // STRING
            data.readShort(); // discard 2 bytes
            break;
          case 3:  // INTEGER
          case 4:  // FLOAT
          case 9:  // FIELD_REF
          case 10: // METHOD_REF
          case 11: // INTERFACE_METHOD_REF
          case 12: // NAME_AND_TYPE
            data.readInt(); // discard 4 bytes;
            break;
          default:
            throw new RuntimeException("Bad tag " + tag);
        }
      }
      short access_flags = data.readShort();
      int this_class = data.readShort();
      int super_class = data.readShort();
      return classNameTable.get(
        offsetTable.get(this_class)).replace('/', '.');
    } catch(Exception e) {
      throw new RuntimeException(e);
    }
  }
  // Demonstration:
  public static void main(String[] args) throws Exception {
    if(args.length > 0) {
```

1101

```
        for(String arg : args)
          print(thisClass(BinaryFile.read(new File(arg))));
      } else
        // Walk the entire tree:
        for(File klass : Directory.walk(".", ".*\\.class"))
          print(thisClass(BinaryFile.read(klass)));
    }
  } ///:~
```

虽然无法在这里介绍其中所有的细节，但每个类文件都必须遵循一定的格式，而我已经尽量用有意义的域名字来表示这些从**ByteArrayInputStream**中提出取来的数据片断。通过施加在输入流上的读操作，你能看出每个信息片的大小。例如，每个类文件的头32个bit总是一个“神秘的数字”hex0xcafebabe[⊖]，而接下来的两个**short**值是版本信息。常量池包含了程序中的常量，所以这是一个可变的值。接下来的**short**告诉我们这个常量池有多大，然后我们为其创建一个尺寸合适的数组。常量池中的每一个元素，其长度可能是一个固定的值，也可能是可变的值，因此我们必须检查每一个常量起始的标记，然后才能知道该怎么做，这就是**switch**语句中的工作。我们并不打算精确地分析类中的所有数据，仅仅是从文件的起始一步一步地走，直到取得我们所需的信息，因此你会发现，在这个过程中我们丢弃了大量的数据。关于类的信息都保存在**classNameTable**和**offsetTable**中。在读完了常量池之后，就找到了**this_class**信息，这是**offsetTable**中的一个坐标，通过它能够找到一个进入**classNameTable**的坐标，然后就可以得到我们所需的类的名字了。 1102

现在，让我们回到**AtUnit.java**程序，**process()**方法现在拥有了类的名字，然后检查它是否包含“.”，如果有就表示该类定义于一个包中。没有包的类将被忽略。如果一个类在包中，那么我们就可以使用标准的类加载器并通过**Class.forName()**将其加载进来。现在，我们终于可以开始对这个类进行**@Unit**注解的分析工作了。

我们只需关心三件事情：首先是**@Test**方法，它们将被保存在**TestMethos**列表中，然后检查是否具有**@TestObjectCreate**和**@TestObjectCleanup**方法。从代码中可以看到，我们通过调用相应的方法来查询注解从而找到这些方法。

每当找到一个**@Test**方法，就打印出当前的类的名字，于是观察者立刻就可以知道发生了什么。接下来开始执行测试，也就是打印出方法名，然后调用**createTestObject()**（如果存在一个加了**@TestObjectCreate**注解的方法），或者调用默认的构造器。一旦创建出测试对象，就调用其上的测试方法。如果测试返回一个**boolean**值，就捕获该结果。如果测试方法没有返回值，那么当没有异常发生时，我们就假设测试成功，反之，如果当**assert**失败或有任何异常抛出时，就说明测试失败，这时将异常信息打印出来以显示错误的原因。如果有失败的测试发生，那么还要统计失败的次数，并将失败的测试所属的类和方法的名字加入**failedTests**，以便最后将其报告给用户。 1103

练习11：(5) 向**@Unit**中加入一个**@TestNote**注解，以便这些附加的信息在测试时能够显示出来。

20.5.4 移除测试代码

对许多项目而言，在发布的代码中是否保留测试代码并没什么区别（特别是在如果你将所有的测试方法都声明为**private**的情况下，如果你喜欢就可以这么做），但是在有的情况下，我们确实希望将测试代码清除掉，精简发布的程序，或者就是不希望测试代码暴露给客户。

⊖ 关于这个数字有许多传说，不过考虑到Java是由书呆子创造出来的，我们可以做一个合理的猜测，他可能正幻想着咖啡店中的某个女人。

与自己动手删除测试代码相比，这需要更复杂的字节码工程。不过开源的Javassist工具类库[⊖]将字节码工程带入了一个可行的领域。下面的程序接受一个**-r**标志作为其第一个参数；如果你提供了该标志，那么它就会删除所有的**@Test**注解，如果你没有提供该标记，那它则只会打印出**@Test**注解。这里同样使用**ProcessFiles**来遍历你选择的文件和目录：

```
//: net/mindview/atunit/AtUnitRemover.java
// Displays @Unit annotations in compiled class files. If
// first argument is "-r", @Unit annotations are removed.
// {Args: ..}
// {Requires: javassist.bytecode.ClassFile;
// You must install the Javassist library from
// http://sourceforge.net/projects/jboss/ }
package net.mindview.atunit;
import javassist.*;
import javassist.expr.*;
import javassist.bytecode.*;
import javassist.bytecode.annotation.*;
import java.io.*;
import java.util.*;
import net.mindview.util.*;
import static net.mindview.util.Print.*;
public class AtUnitRemover
implements ProcessFiles.Strategy {
  private static boolean remove = false;
  public static void main(String[] args) throws Exception {
    if(args.length > 0 && args[0].equals("-r")) {
      remove = true;
      String[] nargs = new String[args.length - 1];
      System.arraycopy(args, 1, nargs, 0, nargs.length);
      args = nargs;
    }
    new ProcessFiles(
      new AtUnitRemover(), "class").start(args);
  }
  public void process(File cFile) {
    boolean modified = false;
    try {
      String cName = ClassNameFinder.thisClass(
        BinaryFile.read(cFile));
      if(!cName.contains("."))
        return; // Ignore unpackaged classes
      ClassPool cPool = ClassPool.getDefault();
      CtClass ctClass = cPool.get(cName);
      for(CtMethod method : ctClass.getDeclaredMethods()) {
        MethodInfo mi = method.getMethodInfo();
        AnnotationsAttribute attr = (AnnotationsAttribute)
          mi.getAttribute(AnnotationsAttribute.visibleTag);
        if(attr == null) continue;
        for(Annotation ann : attr.getAnnotations()) {
          if(ann.getTypeName()
             .startsWith("net.mindview.atunit")) {
            print(ctClass.getName() + " Method: "
              + mi.getName() + " " + ann);
            if(remove) {
              ctClass.removeMethod(method);
              modified = true;
            }
          }
        }
      }
      // Fields are not removed in this version (see text).
      if(modified)
```

1104

⊖ 感谢Shigeru Chiba博士创建了该工具，以及他对我开发**AtUnitRemover.java**的帮助。

```
            ctClass.toBytecode(new DataOutputStream(
              new FileOutputStream(cFile)));
          ctClass.detach();
        } catch(Exception e) {
          throw new RuntimeException(e);
        }
      }
    } ///:~
```

1105

ClassPool是一种全景，它记录了你正在修改的系统中的所有的类，并能够保证所有类在修改后的一致性。你必须从**ClassPool**中取得每个**CtClass**，这与使用类加载器和**Class.forName()**向JVM加载类的方式类似。

CtClass包含的是类对象的字节码，你可以通过它取得类有关的信息，并且操作类中的代码。在这里，我们调用**getDeclaredMethods()**（与Java的反射机制一样），然后从每个**CtMethod**对象中取得一个**MethodInfo**对象。通过该对象，我们察看其中的注解信息。如果一个方法带有**net.mindview.atunit**包中的注解，就将该方法删除掉。

如果类被修改过了，就用新的类覆盖原始的类文件。

在撰写本书时，Javassist刚刚加入了“删除”功能[⊖]，同时我们发现，删除**@TestProperty**域比删除方法复杂得多。因为，有些静态初始化的操作可能会引用这些域，所以你不能简单地将其删除。因此**AtUnitRemover**的当前版本只删除**@Unit**方法。不过，你应该查看一下Javassist网站的更新，因为删除域的功能以后可能也将实现。与此同时，对于**AtUnitExternalText.java**演示的外部测试方法，可以直接删除测试代码生成的类文件，从而到达删除所有测试的目的。

20.6 总结

注解是Java引入的一项非常受欢迎的补充。它提供了一种结构化的，并且具有类型检查能力的新途径，从而使得程序员能够为代码加入元数据，而不会导致代码杂乱且难以阅读。使用注解能够帮助我们避免编写累赘的部署描述文件，以及其他生成的文件。而Javadoc中的**@deprecated**被**@Deprecated**注解取代的事实也说明，与注释性文字相比，注解绝对更适合用于描述类相关的信息。

1106

Java SE5仅提供了很少的内置注解。这意味着如果你在别处找不到可用的类库，那就只能自己创建新的注解以及相应的处理器。有了**apt**工具的帮助，程序员可以同时编译新产生的源文件，以及简化构建过程，不过就目前的情况看，**mirror** API只能给予你一些基本功能，帮助你找到Java类定义中的元素。正如你已经看到的，Javassist能够用来操作字节码，或者你也可以编写自己的字节码操作工具。

这个状况将来一定会改善，API提供方，以及各种framework一定会将注解包含在其提供的工具集内。通过**@Unit**系统，我们可以想像得到，注解很可能将引发Java编程体验的巨大改变。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。

1107～1108

⊖ 应我们的请求，Shigeru Chiba博士将**CtClass.removeMethod()**加入了其中。